

Artificial Neural Network for Diabetes Prediction

Name: Zeyneb Jaghmoun

Course: Data Mining

University: Tunis Business School

Date: April 2026

1. Introduction

In modern healthcare, predicting diseases using data has become essential for early diagnosis and prevention. Machine learning techniques, especially Artificial Neural Networks (ANN), are widely used due to their ability to model complex nonlinear relationships between variables.

In this project, a neural network model (Multilayer Perceptron) is applied to predict whether a patient has diabetes based on medical diagnostic measurements.

[dataset](#)

2. Business Problem

The objective of this project is to predict diabetes occurrence using patient data.

This is a binary classification problem:

- **1** → **Patient has diabetes**
- **0** → **Patient does not have diabetes**

Such predictions can support medical decision-making and early intervention.

3. Dataset Description

The dataset used is the **Pima Indians Diabetes Dataset**.

It contains the following medical variables:

- Pregnancies
- Glucose
- Blood Pressure
- Skin Thickness
- Insulin
- BMI (Body Mass Index)
- Diabetes Pedigree Function

- Age

Target variable:

- **Outcome (0 or 1)**

4. Data Preprocessing

Before training the model, the following steps were applied:

- Separation of features (X) and target (y)
- Splitting data into training (80%) and testing (20%)
- Standardization of features using **StandardScaler**

These steps ensure that all variables are on the same scale, improving model performance and convergence.

5. Model Design

The model used is a **Multilayer Perceptron (MLPClassifier)** from scikit-learn.

Architecture:

- Input layer: 8 features
- Hidden layer 1: 12 neurons (ReLU activation)
- Hidden layer 2: 8 neurons (ReLU activation)
- Output layer: 1 neuron (sigmoid behavior for binary classification)

Other model settings:

- Optimizer: Adam
- Loss function: Log-loss (internal in sklearn)
- Training iterations: max_iter = 500

6. Training Process

The model was trained using supervised learning.

It learns by adjusting weights through backpropagation to minimize classification error.

Unlike deep learning frameworks (e.g., TensorFlow), the training process here uses iterative optimization controlled by **max_iter**.

7. Results

After training, the model achieved the following performance:

- **Accuracy: 77%**

Confusion Matrix:

[[TN FP]
[FN TP]]

From the output:

- Class 0 (Non-diabetic): better performance
- Class 1 (Diabetic): slightly lower detection performance

Classification Report Summary:

- Precision (Class 0): ~0.81
- Recall (Class 0): ~0.83
- Precision (Class 1): ~0.68
- Recall (Class 1): ~0.65
- Macro F1-score: ~0.74
- Weighted F1-score: ~0.77

8. Interpretation

The model demonstrates good predictive ability with 77% accuracy.

It performs better in identifying non-diabetic patients than diabetic patients, which may indicate a slight class imbalance in the dataset.

Key influencing variables likely include:

- Glucose level
- BMI
- Age

These variables are strongly associated with diabetes risk and contribute significantly to prediction accuracy.

9. Limitations

- Moderate dataset size
- Possible class imbalance
- Some medical features may contain noise or missing relationships
- MLPClassifier is less interpretable than simpler models

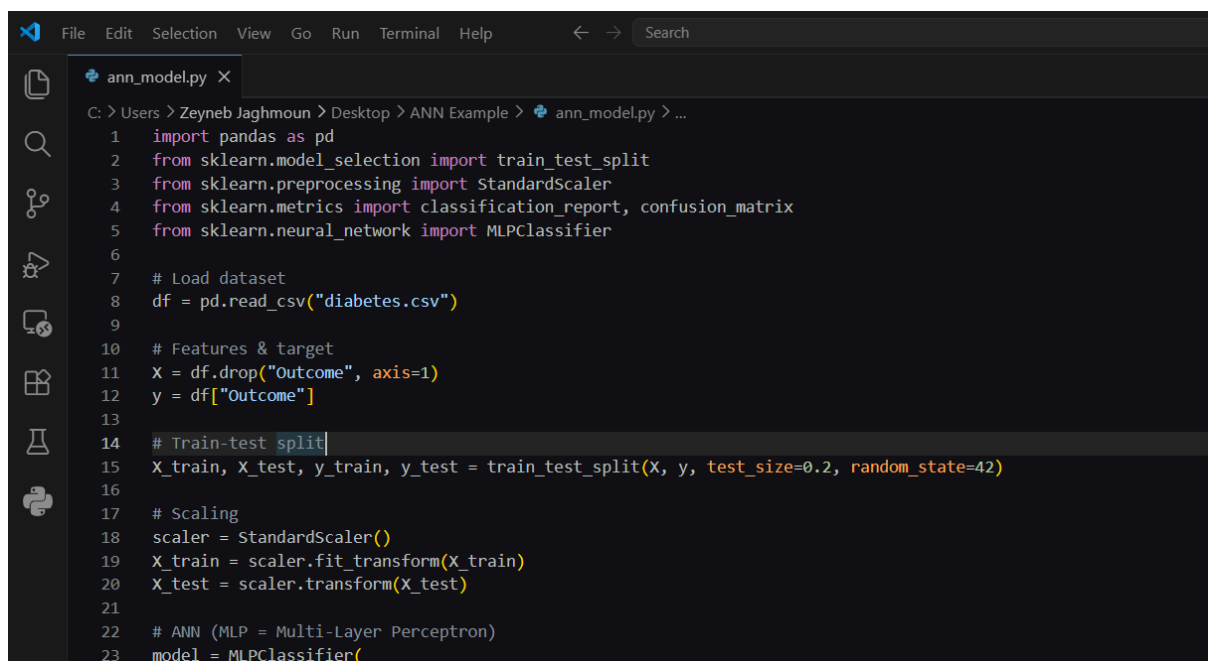
10. Conclusion

This project demonstrates the application of a Multilayer Perceptron model for diabetes prediction using scikit-learn.

The model achieved 77% accuracy and successfully captured nonlinear relationships between medical features.

While performance is solid, further improvements could be achieved through hyperparameter tuning, class balancing, or feature engineering.

11. Appendix (Code) :

A screenshot of a code editor window with a dark theme. The window title is 'ann_model.py'. The menu bar includes 'File', 'Edit', 'Selection', 'View', 'Go', 'Run', 'Terminal', and 'Help'. The file explorer on the left shows icons for file operations. The code is as follows:

```
C: > Users > Zeyneb Jaghmoun > Desktop > ANN Example > ann_model.py > ...
1  import pandas as pd
2  from sklearn.model_selection import train_test_split
3  from sklearn.preprocessing import StandardScaler
4  from sklearn.metrics import classification_report, confusion_matrix
5  from sklearn.neural_network import MLPClassifier
6
7  # Load dataset
8  df = pd.read_csv("diabetes.csv")
9
10 # Features & target
11 X = df.drop("Outcome", axis=1)
12 y = df["Outcome"]
13
14 # Train-test split
15 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
16
17 # Scaling
18 scaler = StandardScaler()
19 X_train = scaler.fit_transform(X_train)
20 X_test = scaler.transform(X_test)
21
22 # ANN (MLP = Multi-Layer Perceptron)
23 model = MLPClassifier()
```

```
File Edit Selection View Go Run Terminal Help
ann_model.py X
C: > Users > Zeyneb Jaghmoun > Desktop > ANN Example > ann_model.py > ...
19 X_train = scaler.fit_transform(X_train)
20 X_test = scaler.transform(X_test)
21
22 # ANN (MLP = Multi-Layer Perceptron)
23 model = MLPClassifier(
24     hidden_layer_sizes=(12, 8), # same idea as your ANN layers
25     activation='relu',
26     solver='adam',
27     max_iter=500,
28     random_state=42
29 )
30
31 # Train
32 model.fit(X_train, y_train)
33
34 # Predict
35 y_pred = model.predict(X_test)
36
37 # Results
38 print(confusion_matrix(y_test, y_pred))
39 print(classification_report(y_test, y_pred))
```

Output :

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python Debug Console
PS C:\Users\Zeyneb Jaghmoun\Desktop\ANN Example> & 'C:\Users\Zeyneb Jaghmoun\AppData\Local\Programs\Python\Python313\python.exe' 'C:\Users\Zeyneb Jaghmoun\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '60914' '-' 'C:\Users\Zeyneb Jaghmoun\Desktop\ANN Example\ann_model.py'
[19 36]]
precision recall f1-score support
0 0.81 0.83 0.82 99
1 0.68 0.65 0.67 55
accuracy 0.75 0.74 0.77 154
macro avg 0.75 0.74 0.74 154
weighted avg 0.76 0.77 0.77 154
PS C:\Users\Zeyneb Jaghmoun\Desktop\ANN Example>
```